

# Python

07

## Методи множин

|                                                      |                                                                                                |
|------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>s.add(x)</b>                                      | Додає елементи <b>x</b> до множини <b>s</b> , якщо вони відсутні в <b>s</b>                    |
| <b>s.clear()</b>                                     | Видаляє всі елементи з множини <b>s</b>                                                        |
| <b>s.copy()</b>                                      | Повертає поверхневу копію множини <b>s</b>                                                     |
| <b>s.difference(t)</b><br><b>s - t</b>               | Повертає нову множину, що включає елементи множини <b>s</b> , які відсутні у множині <b>t</b>  |
| <b>s.difference_update(t)</b><br><b>s -= t</b>       | Видаляє з множини <b>s</b> всі елементи, присутні у множині <b>t</b>                           |
| <b>s.discard(x)</b>                                  | Видаляє елемент <b>x</b> із множини <b>s</b> , якщо він присутній у множині <b>s</b>           |
| <b>s.intersection(t)</b><br><b>s &amp; t</b>         | Повертає нову множину, що включає елементи, присутні одночасно у множинах <b>s</b> та <b>t</b> |
| <b>s.intersection_update(t)</b><br><b>s &amp;= t</b> | Залишає у множині <b>s</b> перетин множин <b>s</b> і <b>t</b>                                  |
| <b>s.isdisjoint(t)</b>                               | Повертає <b>True</b> , якщо множини <b>s</b> і <b>t</b> не мають спільних елементів            |

|                                                  |                                                                                                                                                                                                                               |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>s.issubset(t)</b><br><b>s &lt;= t</b>         | Повертає <b>True</b> , якщо множина <b>s</b> еквівалентна множині <b>t</b> або є його підмножиною; щоб перевірити, чи є множна <b>s</b> тільки підмножиною множини <b>t</b> , слід використовувати перевірку <b>s &lt; t</b>  |
| <b>s.issuperset(t)</b><br><b>s &gt;= t</b>       | Повертає <b>True</b> , якщо множина <b>s</b> еквівалентна множині <b>t</b> або є його надмножиною; щоб перевірити, чи є множина <b>s</b> тільки надмножиною множини <b>t</b> , слід використовувати перевірку <b>s &gt; t</b> |
| <b>s.pop()</b>                                   | Повертає та видаляє випадковий елемент множини <b>s</b> або збуджує виняток <b>KeyError</b> , якщо <b>s</b> – це порожня множина                                                                                              |
| <b>s.remove(x)</b>                               | Видаляє елемент <b>x</b> із множини <b>s</b> або збуджує виняток <b>KeyError</b> , якщо елемент <b>x</b> відсутній у множині <b>s</b>                                                                                         |
| <b>s.symmetric_difference(t)</b><br><b>s ^ t</b> | Повертає нову множину, що включає усі елементи, присутні у множинах <b>s</b> і <b>t</b> , за виключення елементів, присутніх в обох множинах одночасно                                                                        |
| <b>s.union(t)</b><br><b>s   t</b>                | Повертає нову множину, що включає усі елементи множини <b>s</b> і усі елементи множини <b>t</b> , відсутні у множині <b>s</b>                                                                                                 |

# Що таке функція?

Функція в **Python** - це об'єкт приймає аргументи і повертає значення

Для визначення функції необхідно вказати ключове слово **def**

## Навіщо потрібні функції?

Використання функцій дозволяє уникнути дублювання коду, а також зменшити займаний простір. Крім того використання функцій дозволяє структурувати код та покращити його читальність.

# Правила іменування функції

Використовувати у назві функції необхідно нотації **snake\_case** - (def hello\_world)

Також:

1. Не можна починати назву функції з цифр, але використовувати як такі символи їх можна - (def hello\_8)
2. Не можна називати функції зарезервованим ключовим словом - (for, class, def, if)
3. Не можна використовувати у назві функції спеціальні символи, крім нижнього підкреслення - (! # @ % & \* < { } ...)

# Приклади функцій

Ключове слово **return** каже, що потрібно повернути деяке значення. У цьому випадку суму **a + b**

```
1 def add(a, b):  
2     return a + b
```

```
1 >> add(1, 11)  
2 12
```

```
1 >> add('abc', 'def')  
2 'abcdef'
```

```
1 def welcome(name)  
2     return f'You are welcome, {name}!'  
3  
4 welcome('Pavel')  
5  
6 You are welcome, Pavel!
```